

Software Requirements Specification

for

General Issue & Complaint Tracking System

Version 1.0

Prepared By:

Mayank Goyal 24293916114
Vansh Sehrawat 24293916089
Prateek Binjola 24293916071
Deepanshu Sharma 24293916082
Krishan Kumar 24293916088

January 27, 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions, Acronyms, and Abbreviations	3
1.4	References	3
1.5	Overview	4
2	Overall Description	5
2.1	Product Perspective	5
2.2	Product Functions	5
2.3	User Characteristics	5
2.4	General Constraints	6
2.5	Assumptions and Dependencies	6
3	Specific Requirements	7
3.1	External Interface Requirements	7
3.1.1	User Interfaces	7
3.1.2	Hardware Interfaces	7
3.1.3	Software Interfaces	7
3.1.4	Communications Interfaces	7
3.2	Functional Requirements	7
3.2.1	Complaint Submission	7
3.2.2	Real-Time Status Update	8
3.2.3	Secure Login	8
3.3	Performance Requirements	8
3.4	Design Constraints	9
3.5	Software System Attributes	9

1 Introduction

1.1 Purpose

The purpose of this document is to clearly define the software requirements for the **General Issue & Complaint Tracking System**. This document outlines the functional and non-functional requirements, external interfaces, and design constraints to guide the development team and provide stakeholders with a comprehensive understanding of the system's capabilities. The intended audience includes software architects, developers, testers, and project managers.

1.2 Scope

The **General Issue & Complaint Tracking System** is a web-based **Complaint Management Portal** designed to streamline the process of reporting, tracking, and resolving issues. The network-based application allows users to register complaints with supporting evidence (images) and track their status in real-time. For administrators, it provides a centralized dashboard to manage workflows, update statuses, and communicate with users.

Key benefits include:

- Efficient issue reporting and evidence submission.
- Real-time transparency for users regarding complaint status.
- Streamlined workflow management for support teams.

1.3 Definitions, Acronyms, and Abbreviations

MERN	MongoDB, Express.js, React.js, Node.js (The technology stack).
JWT	JSON Web Token (Used for stateless authentication).
ODM	Object Data Modeling (Mongoose library).
API	Application Programming Interface.
REST	Representational State Transfer.
SRS	Software Requirements Specification.
WSS	WebSocket Secure.
HTTP	HyperText Transfer Protocol.

1.4 References

- IEEE Std 830-1984, IEEE Guide to Software Requirements Specifications.

1.5 Overview

The remainder of this document describes the system in detail. Section 2 provides a high-level overview of the product perspective, functions, and users. Section 3 details the specific requirements, including interfaces, functional logic, compliance, and attributes necessary for the development of the MERN-based application.

2 Overall Description

2.1 Product Perspective

The General Issue & Complaint Tracking System is a self-contained web application. It operates on a Client-Server architecture:

- **Client Side:** A browser-based Single Page Application (SPA) built with React.js.
- **Server Side:** A Node.js and Express.js backend handling API requests and business logic.
- **Database:** MongoDB serving as the NoSQL data store.

The system interacts with a file storage system for handling evidence uploads and facilitates real-time communication via WebSockets.

2.2 Product Functions

The system provides distinct functionality for two primary user roles:

- **General User Functions:**
 - User Registration and Secure Login.
 - Submit new complaints with text descriptions and image attachments.
 - View dashboard of submitted complaints with current status.
 - Receive real-time updates when a complaint status changes.
- **Support Admin Functions:**
 - Secure Admin Login.
 - View global dashboard of all complaints.
 - Filter and search complaints.
 - Update complaint status (e.g., specific tags like "In Progress", "Resolved").

2.3 User Characteristics

- **General Users:** Members of the public or customers. Expected to have basic internet proficiency. They require a simple, intuitive interface with clear feedback mechanisms.
- **Support Admins:** Customer support personnel. They possess moderate technical skills and require efficient tools for data management and quick status updates.

2.4 General Constraints

- **Hardware:** The server must be capable of running the Node.js runtime environment. Clients require a device with a modern web browser (Chrome, Firefox, Edge, Safari).
- **Software Stack:** The system must be built using the MERN stack (MongoDB, Express.js, React.js, Node.js).
- **Network:** Reliable internet connection is deemed necessary for all operations.

2.5 Assumptions and Dependencies

- It is assumed that an instance of MongoDB (local or cloud-based like Atlas) is available.
- Users rely on standard HTTP/HTTPS and WebSocket protocols ensuring firewall permissions are granted for these connections.
- Browser support for JavaScript is enabled on client devices.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- The system shall provide a responsive Web UI adaptable to desktop and mobile screens.
- **Dashboard:** A dynamic dashboard using React components to display lists of complaints.
- **Forms:** Clean input forms with validation feedback for registration and complaint submission.

3.1.2 Hardware Interfaces

No specific hardware interface is required beyond the standard network interface cards (NIC) present on the hosting server and client devices to support TCP/IP communication.

3.1.3 Software Interfaces

- **Operating System:** Cross-platform compatibility (Windows, Linux, macOS) supported by the Node.js runtime.
- **Database:** MongoDB (NoSQL Document-oriented database). Interaction managed via Mongoose ODM.
- **Libraries & Frameworks:**
 - Backend: Express.js for routing, Multer for file handling, Bcryptjs for encryption, JSON Web Token (JWT) for auth.
 - Frontend: React.js, Axios for HTTP requests, Socket.io-client for real-time connection.

3.1.4 Communications Interfaces

- **HTTP/HTTPS:** RESTful API endpoints for CRUD operations using JSON format.
- **WebSocket:** Socket.io protocol for bi-directional, low-latency communication between server and client for status updates.

3.2 Functional Requirements

3.2.1 Complaint Submission

Description: Allows authenticated users to report an issue.

- **Input:** Complaint Title, Description, Category, Image File (Optional).
- **Processing:**

1. Client sends **FormData** (multipart/form-data) to the server.
 2. **Multer** middleware intercepts to save the file and generate a path.
 3. **Joi** or **express-validator** validates text fields (e.g., non-empty, max length).
 4. **Mongoose** saves the document (Text Data + Image Path) to MongoDB.
- **Output:** JSON response conforming success (HTTP 201) with the created Ticket ID, or error message (HTTP 400).

3.2.2 Real-Time Status Update

Description: Instantly reflects status changes on the user's dashboard without a page reload.

- **Input:** Admin selects a complaint and changes status (e.g., from 'Open' to 'Resolved').
- **Processing:**
 1. Admin API request updates the status in MongoDB.
 2. Server triggers a **Socket.io** event: `io.emit('status_updated', { ticketId, newStatus })`.
 3. React Frontend listens for the event and updates the state of the specific ticket in the UI list.
- **Output:** The User's browser UI updates the status badge color and text instantly.

3.2.3 Secure Login

Description: Authenticates users and admins securely.

- **Input:** Email Address and Password.
- **Processing:**
 1. Server looks up the user by email in MongoDB.
 2. **Bcryptjs** compares the input password with the stored hashed password.
 3. If match is successful, server signs a **JWT** containing the User ID and Role.
- **Output:** Server returns the JWT string. Client stores this token (localStorage/cookie) for subsequent authenticated requests.

3.3 Performance Requirements

- **Response Time:** API responses should be generated within 500ms under normal load.
- **Real-Time Latency:** WebSocket updates should propagate to clients within 200ms.
- **Concurrency:** The system shall support multiple concurrent users submitting complaints simultaneously without data corruption.

3.4 Design Constraints

- **Technology Stack:** Structure must strictly adhere to the MERN stack.
- **Data Format:** All API exchanges (except file uploads) must use JSON.
- **Compliance:** Code structure should follow standard MVC (Model-View-Controller) or Service-oriented architectural patterns suitable for Express apps.

3.5 Software System Attributes

- **Security:**
 - Passwords must never be stored in plain text (Bcrypt enforcement).
 - API endpoints must be protected via JWT verification middleware.
 - File uploads must be validated for type (e.g., images only) to prevent execution of malicious scripts.
- **Availability:** The system should aim for 99.9% uptime during business hours.
- **Maintainability:** Usage of separate definition schemas (Mongoose models) and modular route handlers to ensure code is easy to read and maintain.